

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: SEPTEMBER 10 2026

DATE SUBMITTED: SEPTEMBER 10 2026

NAME: Medina, Jomz Ryle E., Mendoza, Roy Gabriel I., Mauro, Ryan Lewis B., Mendoza, Ryo Shin

INSTRUCTOR: Engr. Jimlord Quejado

SEATWORK NO. 8.1 USING SORTING ALGORITHMS 2

1. PROCEDURE OUTPUT

Objectives: To create a program in C++ using sorting algorithms.

1. To be familiarized with the sorting algorithms.
2. To be able to differentiate the types of sorting algorithms.
3. To be able to create a program with sorting algorithms.

Answer the following questions:

1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.

QUICK SORT:

> **Mauro**

> Quick Sort is a sorting algorithm that chooses one element as a pivot and divides the other elements into two groups: elements smaller than the pivot and elements larger than the pivot. It then sorts the two groups repeatedly until the entire list is sorted. It is generally fast and works well with large amounts of data. The process continues by selecting another pivot from each group and dividing the elements again. This makes the groups smaller and easier to sort. Once all the smaller groups are sorted, they are arranged together to produce the final sorted list, however, its performance can become slower if the pivot is chosen poorly.

SHELL SORT:

> **Ryo Shin**

> It is like an improved version of Insertion Sort, but it is more suitable for larger datasets because it compares elements that are far apart using a **gap**. The gap becomes smaller until it reaches 1, where it finishes sorting elements like Insertion Sort. For the time complexity, its best-case is **$O(n \log n)$** because it allows the elements to move closer to their correct positions faster, reducing the number of comparisons and movement compared to Insertion Sort.

MERGE SORT:

> **Mendoza, Roy**

> Merge sort is a sorting algorithm that follows the concept of “**Divide and Conquer**” by basically breaking down the array into sub arrays until each has only a single slot or element, then recombining the arrays from the ground up in a sorted way.

The specific process is that it divides the origin array into 2 and that each sub array is divided into 2 until there is only 1 element slot, after that, each almost like next or closest sub array that is near them, they are then compared with each other to see who is the smaller, the smaller side goes to the left(if ascending), then the higher goes to the right. It then works from this concept from each single slot or element until it rebuilds back the origin array into a single sorted array.

In terms of the time complexity, it is an **$O(n \log n)$** in both the best and worst because, we first have the **$\log n$** due to us halving the array in a loop until we divide all the elements into a single sub arrays for each element, we then have the **n** because we scan and merge all the elements into across the different sub arrays levels.

2. Give simple sample programs in C++ that uses the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work.

Medina, Jomz Ryle

1. Shell Sort

```
1 shellSort.cpp > ...
2 #include <iostream>
3 using namespace std;
4
5 const int maxSize = 10;
6
7 // FOLLOWING LAB PSEUDOCODE:
8 /*
9 shellSort(array, size)
10   for interval i <= size/2n down to 1
11     for each interval "i" in array
12       sort all the elements at interval "i"
13   */
14
15 // Function prot
16 void shellSort(int arr[], int size);
17
18 int main() {
19     int array[maxSize];
20
21     cout << "Enter 10 integers:\n";
22     for(int i = 0; i < maxSize; i++){
23         cin >> array[i];
24     }
25
26     shellSort(array, maxSize);
27     cout << "\nSorted:\n";
28
29     for(int i = 0; i < maxSize; i++){
30         cout << array[i] << " ";
31     }
32
33     return 0;
34 }
35
```

```
37 // Function Definition
38 void shellSort(int arr[], int size) {
39     int interval = size / 2;
40
41     while (interval >= 1) {
42         for (int i = interval; i < size; i++) {
43             int temp = arr[i];
44             int j = i;
45
46             while (j >= interval && arr[j - interval] > temp) {
47                 arr[j] = arr[j - interval];
48                 j = j - interval;
49             }
50
51             arr[j] = temp;
52         }
53
54         interval = interval / 2;
55     }
56 }
```

Terminal Output:

```
Enter 10 integers:
10
7
8
9
6
5
2
3
4
1

Sorted:
1 2 3 4 5 6 7 8 9 10
```

Explanation:

The following code above is an example of shell sort where the user is first prompted to enter 10 values before it is then sorted via calling the function for shell sort. This sorting algorithm follows the principle of comparing as well as sorting numbers that are separated by gaps (the interval). At the start, the gap is half the size of the array before being divided by two every round until it turns into what resembles an insertion sort. From there, as the interval becomes 1, the program sorts the array into its final, sorted version.

2. Merge Sort

```
@ mergeSort.cpp > merge(int [], int, int, int)
1  #include <iostream>
2
3  // FOLLOWING LAB PSEUDOCODE:
4  /*
5  Declare an array Arr of length N
6  If N=1, Arr is already sorted
7  If N>1,
8  Left = 0, right = N-1
9  Find middle = (left + right)/2
10 Call merge_sort(Arr,left,middle) =>sort first half recursively
11 Call merge_sort(Arr,middle+1,right) => sort second half recursively
12 Call merge(Arr, left, middle, right) to merge sorted arrays in above steps.
13 Exit
14 */
15
16 const int maxSize = 10;
17
18 // Function prototypes
19 void merge(int array[], int left, int middle, int right);
20 void mergeSort(int array[], int left, int right);
21
22 int main() {
23     int array[maxSize];
24
25     std::cout << "Enter 10 integers:\n";
26     for(int i = 0; i < maxSize; i++){
27         std::cin >> array[i];
28     }
29
30     mergeSort(array, 0, maxSize - 1);
31
32     std::cout << "\nSorted:\n";
33     for(int i = 0; i < maxSize; i++){
34         std::cout << array[i] << " ";
35     }
36
37     return 0;
38 }

41 // Function Definitions
42 void merge(int array[], int left, int middle, int right) {
43     int temp[maxSize];
44     int i = left;
45     int j = middle + 1;
46     int k = left;
47
48     while(i <= middle && j <= right){
49         if (array[i] < array[j]) {
50             temp[k] = array[i];
51             i++;
52         }
53         else {
54             temp[k] = array[j];
55             j++;
56         }
57         k++;
58     }
59
60     while(i <= middle){temp[k] = array[i]; i++; k++;}
61     while(j <= right){temp[k] = array[j]; j++; k++;}
62
63     for(int x = left; x <= right; x++){
64         array[x] = temp[x];
65     }
66 }
67
68 void mergeSort(int array[], int left, int right) {
69     if(left < right) {
70         int middle = (left + right) / 2;
71         mergeSort(array, left, middle);
72         mergeSort(array, middle + 1, right);
73         merge(array, left, middle, right);
74     }
75 }
76
77 }
```

Explanation:

Much like with the Shell Sort program, we have it asking the user for 10 inputs into an array to sort. Then, it calls merge sort which uses the principle of 'divide and conquer' to split the array into smaller halves before using the 'merge()' function to combine the values in correct order. Then, it just repeats until the array is completely sorted.

Terminal Output:

```
Enter 10 integers:
10
5
6
7
9
8
4
3
1
2

Sorted:
1 2 3 4 5 6 7 8 9 10
```

3. Quick Sort

```
1  #include <iostream>
2
3  const int maxSize = 10;
4
5  // Function Prototypes
6  int partition(int array[], int low, int high);
7  void quickSort(int array[], int low, int high);
8
9  int main() {
10     int array[maxSize];
11
12     std::cout << "Enter 10 integers:\n";
13     for(int i = 0; i < maxSize; i++){
14         std::cin >> array[i];
15     }
16
17     quickSort(array, 0, maxSize - 1);
18
19     std::cout << "\nSorted:\n";
20
21     for(int i = 0; i < maxSize; i++){
22         std::cout << array[i] << " ";
23     }
24
25     return 0;
26 }
```

```
28 // Function Definitions
29 int partition(int array[], int low, int high) {
30     int pivotValue = array[high];
31     int position = low;
32
33     for(int i = low; i < high; i++){
34         if (array[i] < pivotValue) {
35             std::swap(array[i], array[position]);
36             position++;
37         }
38     }
39     std::swap(array[position], array[high]);
40
41     return position;
42 }
43
44 void quickSort(int array[], int low, int high) {
45     if(low < high){
46         int pivot = partition(array, low, high);
47
48         quickSort(array, low, pivot - 1);
49         quickSort(array, pivot + 1, high);
50     }
51 }
```

Explanation:

After the program above receives the 10 numbers from the user and creates an array, the algorithm for quick sort is applied where the last number is used as a pivot where the function, '**partition()**' is called that places the numbers smaller than the pivot to the left of the pivot and numbers larger to the right. The function, '**quicksort()**' then does the same process for the left and right created piles of said pivot until the very end where the array is finally sorted.

Terminal Output:

```
Enter 10 integers:
5
4
3
2
9
8
7
6
1
10

Sorted:
1 2 3 4 5 6 7 8 9 10
```

2. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.